QUY CHUẨN PHÁT TRIỂN WEBSITE TOÀN DIỆN (A - Z)

Tối Ưu Hiệu Suất • Clean Code • Kiến Trúc Doanh Nghiệp Cốt Lõi | Dự Án: Antigravity

Tài liệu này định hình toàn bộ quy trình, tiêu chuẩn kỹ thuật và kiến trúc áp dụng cho dự án chiến lược **Antigravity**. Quy chuẩn bao quát trọn vẹn từ trải nghiệm người dùng đến kiến trúc hạ tầng tầng sâu, đảm bảo hệ thống đạt hiệu năng tối đa, mã nguồn dễ bảo trì, mở rộng và sẵn sàng vận hành trong môi trường enterprise thực tế.

PHẦN 1: QUY TRÌNH PHÁT TRIỂN CHUẨN NĂM GIAI ĐOẠN (PIPELINE)

1. Giai Đoạn UX (User Experience) - Nền Tảng Trải Nghiệm

Trải nghiệm người dùng quyết định sự giữ chân của khách hàng. Giai đoạn này tập trung hoàn toàn vào luồng đi của dữ liệu và tâm lý học hành vi người dùng trước khi triển khai bất kỳ giao diện trực quan nào.

- **User Research & Persona:** Xác định tối thiểu 2 nhóm đối tượng người dùng mục tiêu. Xây dựng sơ đồ thấu cảm (Empathy Map) nhằm làm rõ nhu cầu cốt lõi.
- **User Flows & Information Architecture (IA):** Bản đồ cấu trúc website không vượt quá 3 cấp độ nhấn chuột sâu (3-click rule) từ Trang chủ đến trang chức năng đích.
- **Wireframing Lo-Fi:** Thiết kế khung xương tập trung vào việc bố trí phân rã bố cục phần tử, phân chia tỷ lệ tương phản thị giác cấp bậc thông tin (Visual Hierarchy).
- **Heuristic Evaluation:** Đáp ứng nghiêm ngặt 10 nguyên lý thiết kế giao diện của Jakob Nielsen, đặc biệt chú trọng trạng thái hệ thống luôn hiển thị phản hồi trực quan rõ ràng trong vòng 200ms.
- **Accessibility (WCAG 2.1):** Thiết kế tuân thủ bộ tiêu chuẩn AA. Độ tương phản chữ viết đạt tỷ lệ tối thiểu 4.5:1. Hỗ trợ điều hướng hoàn toàn bằng phím Tab cho mọi chức năng tương tác động.

2. Giai Đoạn UI (User Interface) - Thiết Kế Hệ Thống Đồng Bộ

Chuyển đổi khung xương UX thành giao diện trực quan sống động thông qua phương pháp Component-Driven Design nhằm tăng tốc tính tái sử dụng và đảm bảo tính nhất quán.

- **Design System & Design Tokens:** Định nghĩa bộ quy chuẩn tập trung về mã màu, kích thước chữ (Typography), khoảng cách padding/margin (Spacing), hiệu ứng đổ bóng thông qua hệ thống Token cấu trúc biến nhất quán, ánh xạ trực tiếp sang cấu hình Tailwind CSS.

- **Dark Mode Specification:** Tất cả thành phần giao diện bắt buộc phải có biến thể Dark Mode tương ứng. Sử dụng dải màu xám sâu/slate đậm (ví dụ: `#0f172a` hoặc `#1e293b`) làm nền, tránh dùng màu đen tuyệt đối (`#000000`) để giảm thiểu mỏi mắt.
- **Responsive Layout Grid:** Sử dụng hệ thống lưới 12 cột (12-column grid) linh hoạt. Định nghĩa rõ 4 mốc Breakpoint tiêu chuẩn: Mobile (<640px), Tablet (640px - 1024px), Desktop (1024px - 1440px), và UltraWide (>1440px).
- **Component States:** Toàn bộ thành phần tương tác (Button, Input, Card) phải được thiết kế chi tiết 5 trạng thái động: Default, Hover, Active, Focus, và Disabled.

3. Giai Đoạn Frontend (FE) - Kiến Trúc & Tối Ưu Hóa Trình Diễn

Xây dựng giao diện hiệu năng cao ứng dụng công nghệ Next.js/React tiên tiến. Cấu trúc mã nguồn tổ chức theo mô hình hướng tính năng mạnh mẽ.

Tiêu Chí Quy Chuẩn	Giải Pháp Triển Khai Thực Tế
Kiến Trúc Mã Nguồn	Áp dụng **Feature-driven Architecture**. Gom nhóm toàn bộ components, hooks, api hooks, và types liên quan trực tiếp đến một nghiệp vụ cụ thể vào một thư mục (ví dụ: `/features/auth`, `/features/dashboard`).
Chiến Lược Render	Sử dụng linh hoạt cơ chế Server-Side Rendering (SSR) cho các trang quản trị yêu cầu bảo mật dữ liệu tươi mới, Incremental Static Regeneration (ISR) cho trang tin tức/sản phẩm để đạt tốc độ phản hồi tức thì, và Static Site Generation (SSG) cho trang đích (Landing Page).
Quản Lý Trạng Thái	Tách biệt dữ liệu: Sử dụng **Zustand** cho Global Client State (gọn nhẹ, không gây re-render diện rộng), và dùng **TanStack Query (React Query)** để cache và quản lý toàn bộ Server State từ API.
Tối Ưu Core Web Vitals	Đạt điểm số >90 trên Google Lighthouse. Bắt buộc tối ưu hóa ảnh qua component `next/image`, thiết lập thuộc tính kích thước tường minh để triệt tiêu hiện tượng dịch chuyển bố cục đột ngột (CLS < 0.1). Trì hoãn load mã nguồn không thiết yếu thông qua Dynamic Import (Lazy Loading).

Cấu trúc thư mục Frontend chuẩn hóa ứng dụng Next.js App Router:

```
src/
├── app/                  # App Router (Pages, Layouts, API Routes)
├── components/           # UI Components dùng chung toàn hệ thống (Button, Modal)
├── config/               # Cấu hình môi trường, hằng số, Axios Client
├── features/             # Lõi nghiệp vụ chia theo tính năng độc lập
│   ├── esports-dashboard/
│   │   ├── components/   # Component đặc thù của riêng tính năng dashboard
│   │   ├── hooks/        # Custom hooks xử lý logic hoặc API fetch
│   │   ├── services/     # Hàm gọi API endpoint
│   │   └── types.ts      # Định nghĩa TypeScript của riêng tính năng
├── hooks/                # Global Hooks (useAuth, useTheme, useDebounce)
├── store/                # Cấu hình Zustand Global Store
└── types/                # Type định nghĩa dùng chung toàn hệ thống
```

4. Giai Đoạn Backend (BE) - Kiến Trúc Đóng Gói Tầng Sâu

Xây dựng lõi xử lý nghiệp vụ bền vững bằng Node.js kết hợp TypeScript. Sử dụng kiến trúc phân lớp doanh nghiệp phổ biến nhất (Layered Architecture / Clean Architecture tinh giản) để phân tách rõ ràng trách nhiệm.

> **Quy tắc Kiến trúc Phân lớp Backend Tuân thủ:**
>
> **Controller Layer:** Chỉ chịu trách nhiệm tiếp nhận HTTP Request, bóc tách tham số đầu vào và trả về HTTP Response. Tuyệt đối không chứa logic nghiệp vụ hay câu lệnh truy vấn database trực tiếp tại đây.
>
> **Service Layer (Business Logic):** Nơi thực thi toàn bộ luồng xử lý dữ liệu, kiểm tra quyền hạn, tính toán thuật toán, và phối hợp điều hướng giữa các Repository.
>
> **Repository/Model Layer:** Tầng tương tác trực tiếp duy nhất với cơ sở dữ liệu, đóng gói các câu lệnh truy vấn phức tạp thành hàm gọn gàng.

- **Bảo Mật Hệ Thống:** Triển khai xác thực phi tập trung qua cơ chế Token kép (Access Token thời hạn ngắn 15 phút lưu bộ nhớ tạm, Refresh Token thời hạn dài 7 ngày bọc trong cookie HttpOnly bảo mật cao). Chống tấn công brute force thông qua thư viện cấu hình Rate Limiting.
- **Xử Lý Lỗi Tập Trung (Global Error Handler):** Tự động bắt mọi ngoại lệ thông qua một Middleware xử lý lỗi tập trung duy nhất. Không bao giờ được phép làm sập tiến trình Node.js (tránh unhandled rejections).
- **Định Chuẩn Định Dạng API:** Tất cả kết quả trả về từ REST API bắt buộc phải đồng bộ theo cấu trúc JSON chuẩn hóa như sau:

```
{
  "success": true,
  "message": "Lấy thông tin dữ liệu thành công",
  "data": { ... },
  "meta": { "page": 1, "limit": 10, "total": 120 } // Dành riêng cho dữ liệu phân trang
}
```

5. Giai Đoạn Database (DB) - Thiết Kế & Tối Ưu Hóa Lưu Trữ

Tùy chọn hạ tầng dữ liệu tùy theo tính chất bài toán nghiệp vụ cụ thể. Dự án Antigravity ưu tiên sử dụng MongoDB hoặc cơ sở dữ liệu quan hệ tùy chỉnh phù hợp hiệu năng cao.

- **Indexing Chiến Lược:** Tạo Single Index hoặc Compound Index cho toàn bộ các trường thường xuyên xuất hiện trong mệnh đề tìm kiếm (`where` / `find`) hoặc mệnh đề sắp xếp (`sort`). Tuyệt đối triệt tiêu trạng thái truy vấn quét toàn bộ bảng (Collection/Table Scan).
- **Quy Chuẩn Thiết Kế Schema (MongoDB):** Cân nhắc kỹ bài toán Reference (Tham chiếu ID) và Embedding (Nhúng tài liệu con). Với dữ liệu có tần suất thay đổi thấp và kích thước giới hạn (ví dụ: danh

sách địa chỉ, chi tiết cấu hình sản phẩm), hãy dùng Embedding để giảm thiểu số lượng lệnh Join/Lookup phức tạp, tăng tốc độ đọc dữ liệu cực hạn.

- **Tối Ưu Hóa Truy Vấn:** Giới hạn nghiêm ngặt lượng dữ liệu trả về bằng cách áp dụng Projection chỉ lấy đúng các trường cần hiển thị. Mọi câu lệnh danh sách bắt buộc phải có cơ chế phân trang cứng ở tầng database (sử dụng toán tử `limit` và `skip`).
- **Connection Pooling:** Thiết lập tham số số lượng kết nối tối đa và tối thiểu (Max/Min Pool Size) phù hợp để tối ưu dung lượng tài nguyên máy chủ, tránh gây nghẽn hàng đợi kết nối khi hệ thống chạm mốc tải đỉnh đột biến.

PHẦN 2: CHUẨN CONTAINERIZATION & DEVOP DEPLOYMENT

Bắt buộc đóng gói toàn bộ ứng dụng thành các Container độc lập bằng **Docker** và phối hợp vận hành nhịp nhàng thông qua **Docker Compose**, kết hợp **Nginx** làm Reverse Proxy điều phối lưu lượng.

> **Tiêu chuẩn Dockerfile Tối Ưu (Multi-stage Build):**
>
> Tuyệt đối không đưa toàn bộ mã nguồn thô và bộ cài đặt Node_modules của môi trường phát triển vào container vận hành cuối. Phải chia Dockerfile làm 2 giai đoạn: Giai đoạn 1 (Build Stage) dùng image gốc đầy đủ để biên dịch mã nguồn mã hóa TypeScript; Giai đoạn 2 (Production Stage) chỉ copy sản phẩm sau khi build và các thư viện cần thiết vào một image rút gọn siêu nhẹ dạng **Alpine Linux** để thu nhỏ kích thước image xuống dưới mốc 150MB, giảm thiểu tối đa lỗ hổng bảo mật.

Cấu hình Nginx Reverse Proxy chuẩn hóa điều phối luồng chạy:

```
server {
    listen 80;
    server_name antigravity.local;

    # Cấu hình xử lý Frontend Tốc độ cao
    location / {
        proxy_pass http://frontend:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }

    # Điều hướng luồng cổng API về Cụm Backend
    location /api/v1/ {
        proxy_pass http://backend:5000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

        # Thiết lập thời gian Timeout chống nghẽn đường truyền
        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }
}
```